# JOBSHEET 17

# IMPLEMENTASI LOGIN GOOGLE PROVIDER DENGAN NEXTAUTH.JS + FIREBASE

## PEMROGRAMAN BERBASIS FRAMEWORK


**REIKA AMALIA SYAHPUTRI**

**2341720173**

**TI-3E / 19**

**TEKNOLOGI INFORMASI**

# Github:

[https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_17](https://github.com/ReikaAmalia/Pemrograman_Berbasis_Framework/tree/main/praktikum_17)

# PRAKTIKUM: Google Login dengan NextAuth.js

### A. Persiapan

**Pastikan Project Sudah Memiliki:**

• Next.js (App Router atau Pages Router)

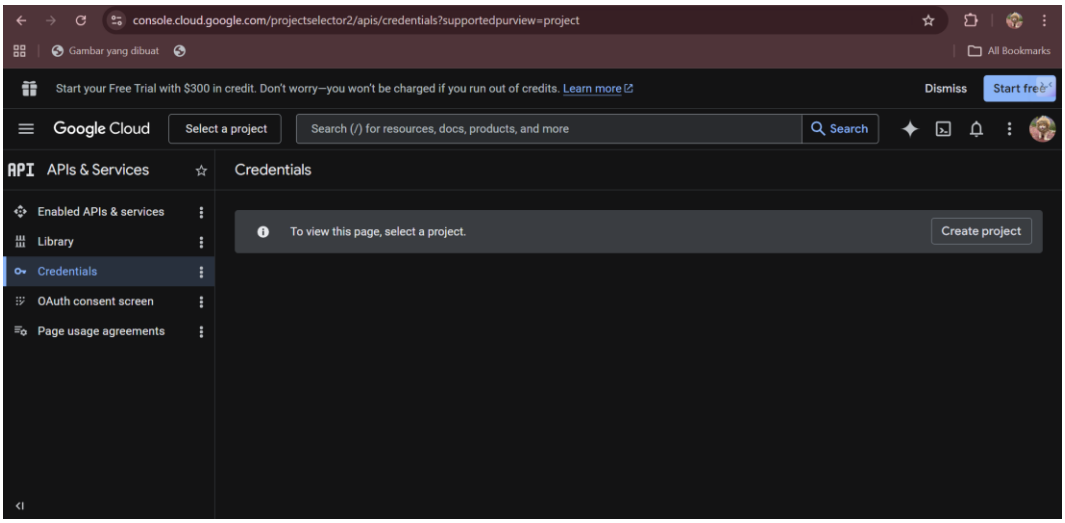
• NextAuth.js terinstall

• Firebase Firestore terkoneksi

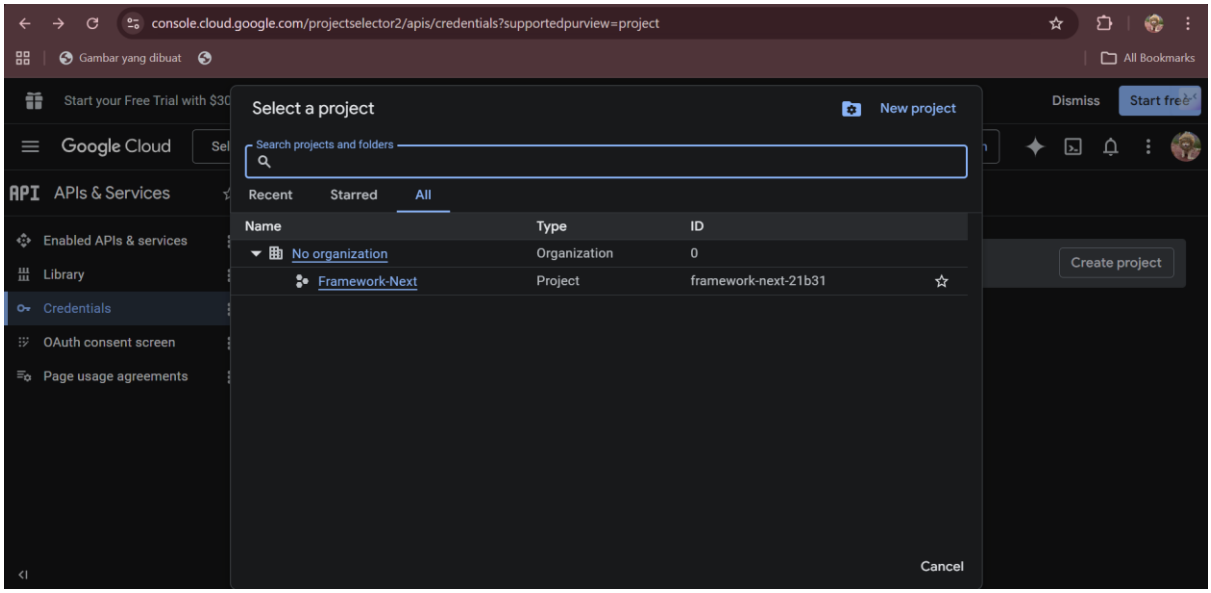
• Sistem login credential sudah berjalan (opsional)

### B. Konfigurasi Google OAuth

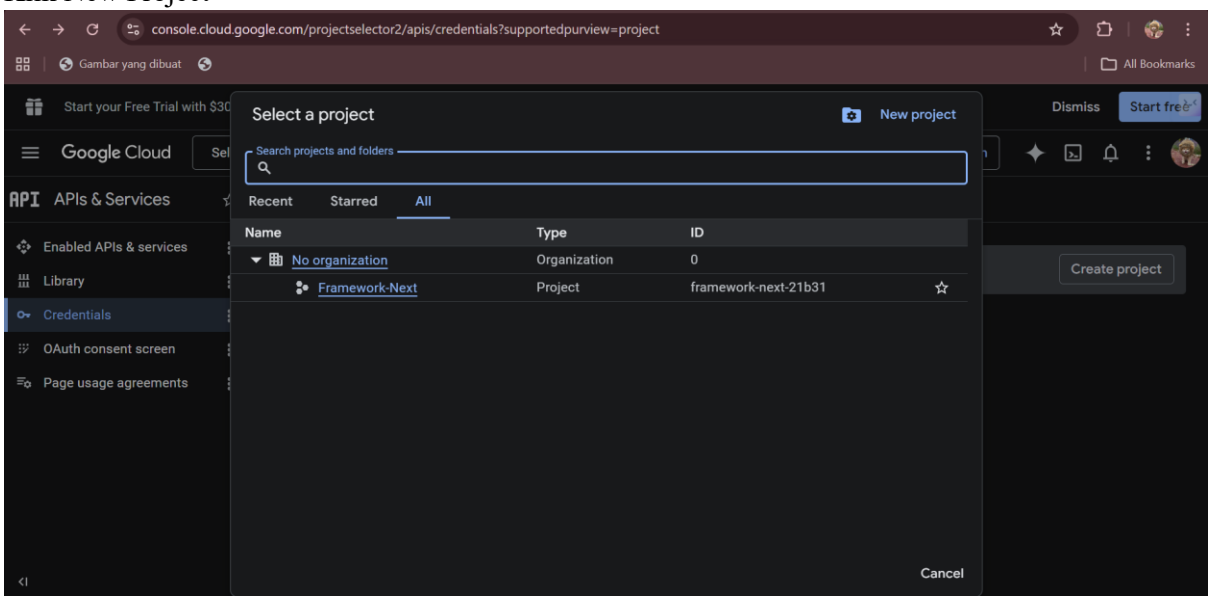
- Langkah 1 – Masuk ke Google Cloud Console Buka:
- https://console.cloud.google.com/apis/credentials



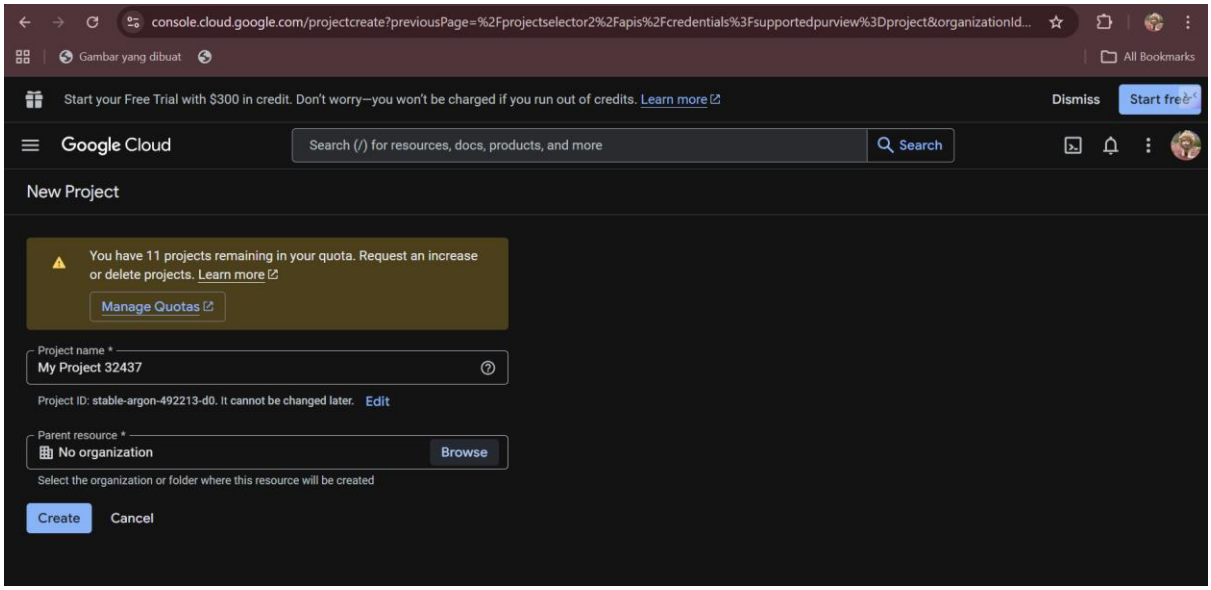

- Langkah 2 – Buat Project Baru



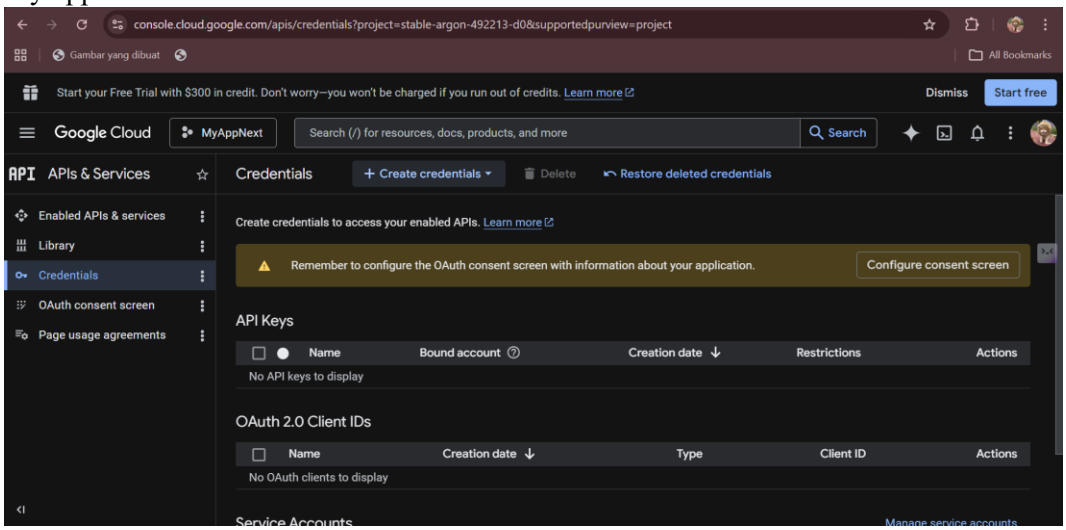

- Klik New Project




- Nama project: MyAppNext

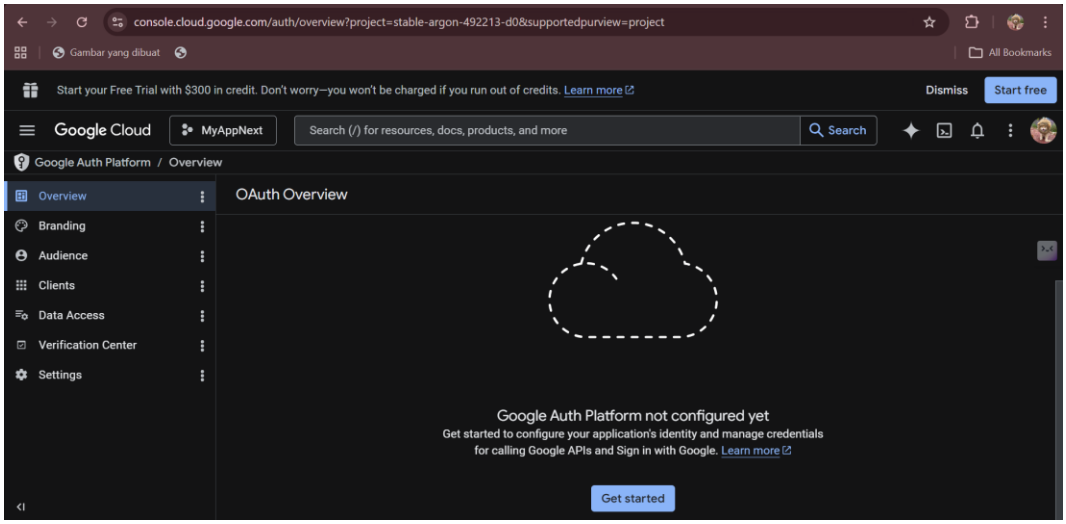



- Klik Create
- Setelah berhasil klik https://console.cloud.google.com/apis/credentials pastikan projectnya MyAppNext

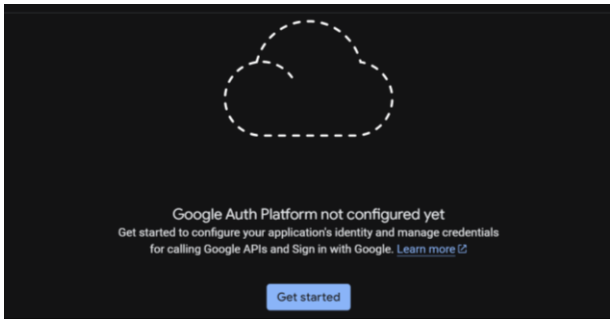



**Langkah 3 – Konfigurasi OAuth Consent Screen**

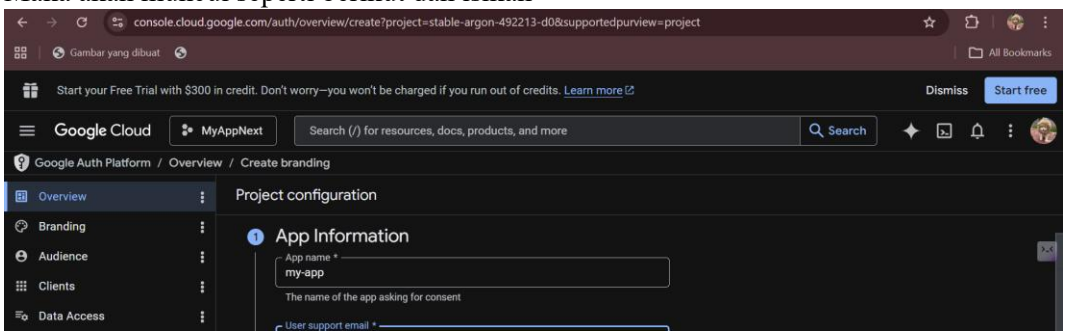
1. Pilih OAuth consent screen

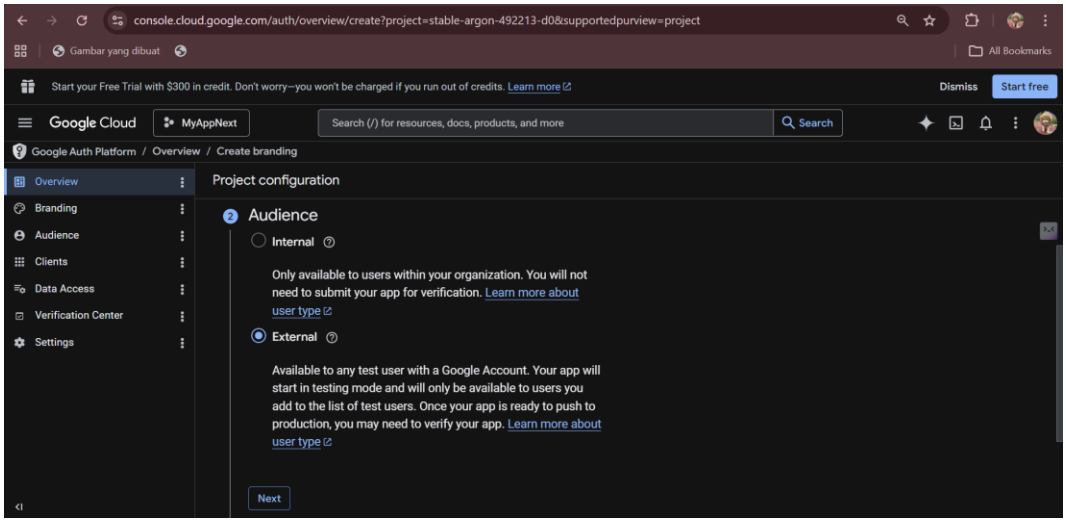


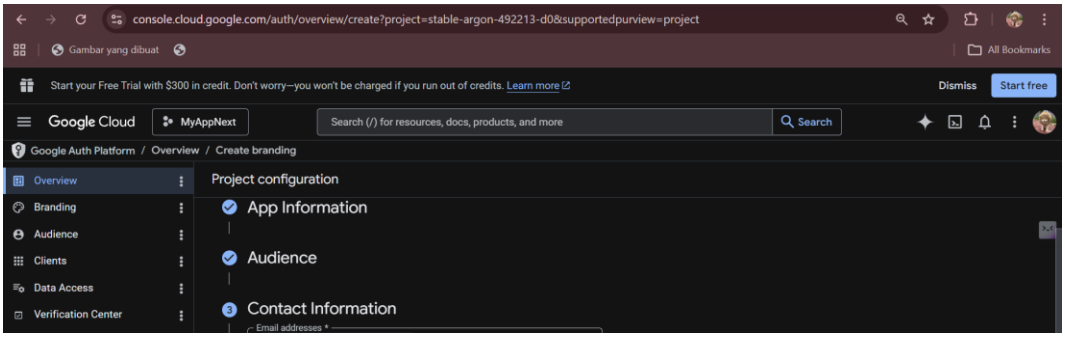

2. Pilih Get Started

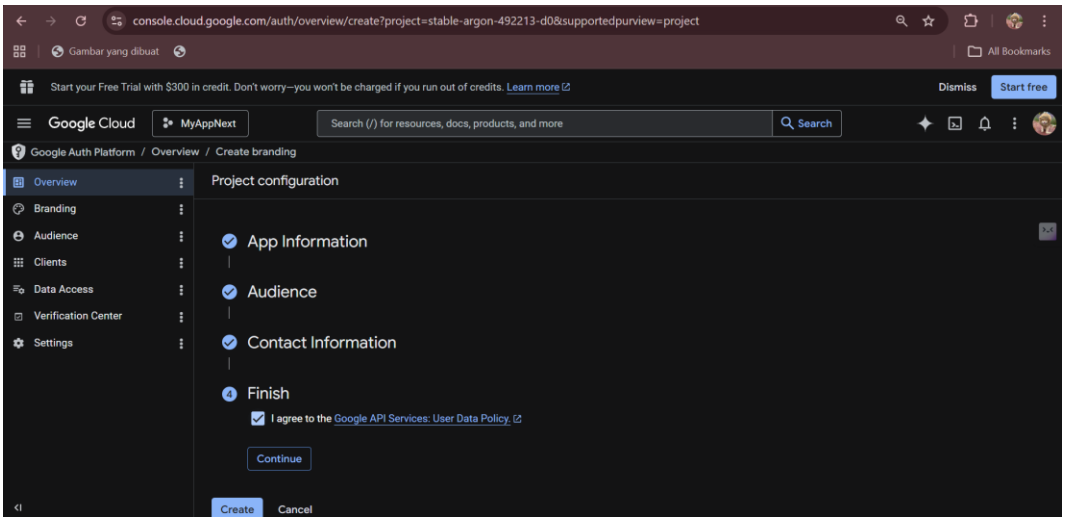



3. Maka akan muncul seperti berikut dan isikan

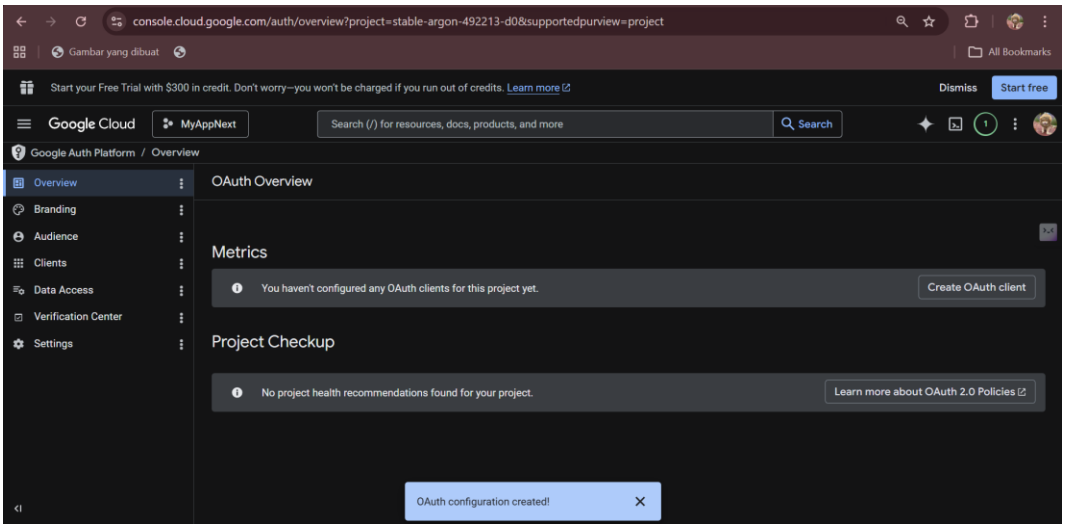




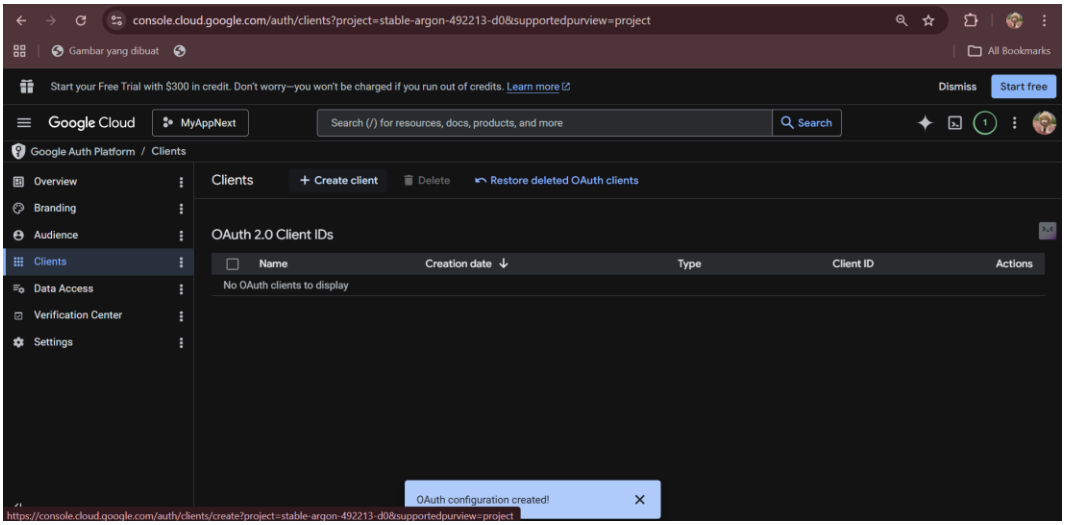



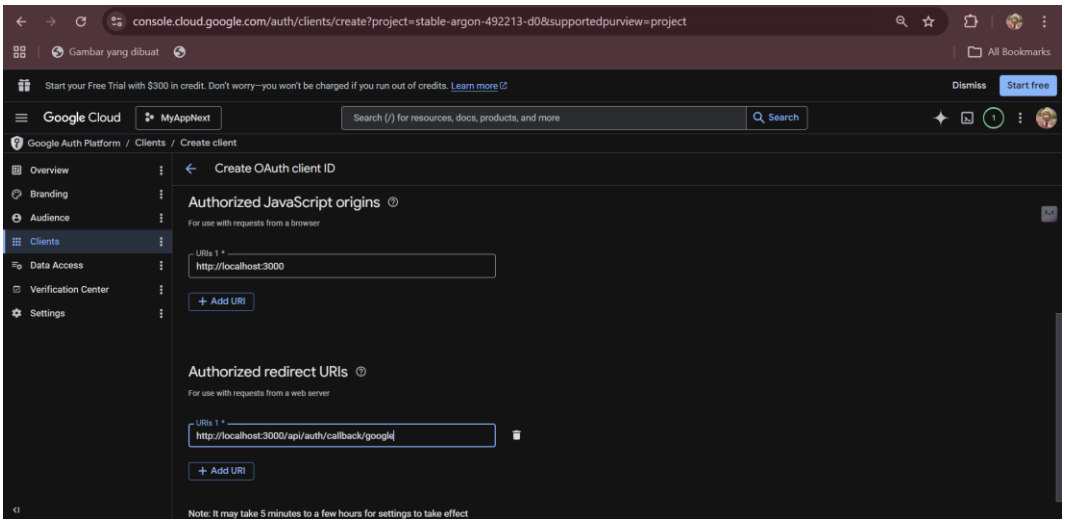

- Klik create




**Langkah 4 – Buat OAuth Credentials**

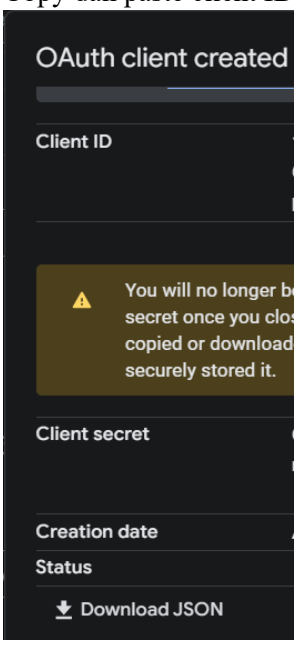
1. Klik create client pada Clients




## C. Tambahkan Environment Variables

1. Copy dan paste client ID dan Client secret ke .env



```
praktikum_17
.env.local
my-app-17 > .env.local
12    # lalu masukkan ke da
13    NEXTAUTH_SECRET=dG9vb
14
15    GOOGLE_CLIENT_ID=1094
16    GOOGLE_CLIENT_SECRET=
```

**D. Konfigurasi Google Provider di NextAuth dan Handle Callback JWT & Session**

1. Buka file [...nextauth].ts pada folder api/auth dan modifikasi menjadi berikut

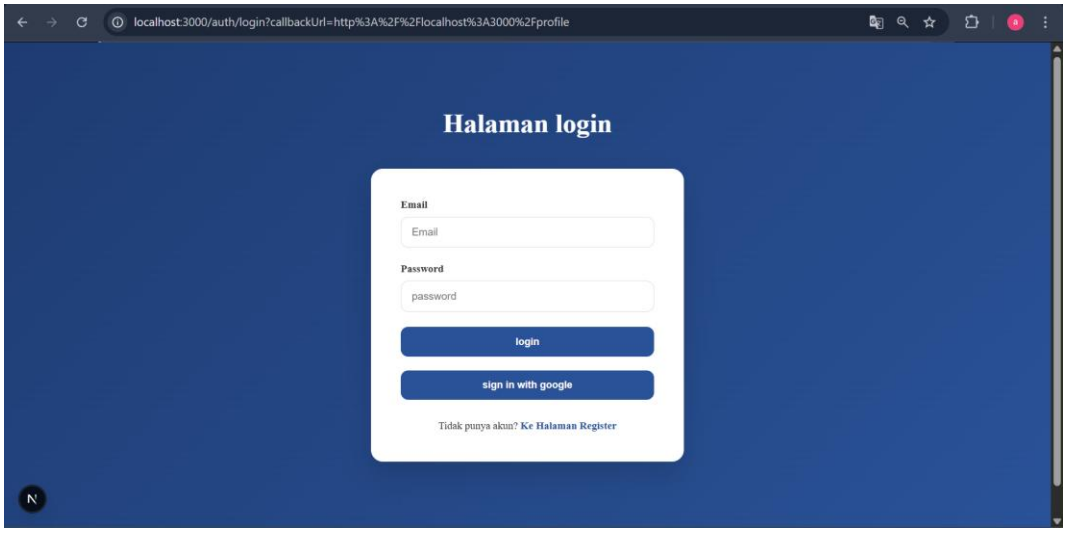
```python
import { signIn } from "@/utils/db/servicefirebase";
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import bcrypt from "bcrypt";
import GoogleProvider from "next-auth/providers/google";

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,

  providers: [
    // credentials provider untuk login
    CredentialsProvider({
      name: "credentials",
      credentials: {
        // fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials.password)  return null;

        const user: any = await signIn(credentials.email);
        if (user) {
          const isPasswordValid = await bcrypt.compare(
            credentials.password,
            user.password,
          );
          if (isPasswordValid) {
            // pastikan mmengembalikan object user yang bersih
            return {
              id: user.id,
              email: user.email,
              fullname: user.fullname,
              role: user.role,
            };
          }
        }
        return null;
      },
    }),
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID || "",
      clientSecret: process.env.GOOGLE_CLIENT_SECRET || "",
    }),
  ],

  callbacks: {
    // async redirect({ url, baseUrl }) {
    //   if (url.startsWith(baseUrl)) return url;
    //   if (url.startsWith("/")) return `${baseUrl}${url}`;
    //   return `${baseUrl}/profile`;
    // },

    async jwt({ token, account, profile, user }: any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
        token.role = user.role;
      }
      if (account?.provider === "google") {
        const data = {
          fullname: user.name,
          email: user.email,
          image: user.image,
          type: account.provider,
        };
        token.fullname = data.fullname;
        token.email = data.email;
        token.image = data.image;
        token.type = data.type;
        }
      return token;
    },
    async session({ session, token }: any) {
      if (token.email) {
        session.user.email = token.email;
      }
      if (token.fullname) {
        session.user.fullname = token.fullname;
      }
      if (token.image) {
        session.user.image = token.image;
      }
      if (token.role) {
        session.user.role = token.role;
      }
      if (token.type) {
        session.user.type = token.type;
      }
      return session;
    },
  },

  // pengarah sign in ke halaman login
  pages: {
    signIn: "/auth/login",
  },
};

export default NextAuth(authOptions);
```

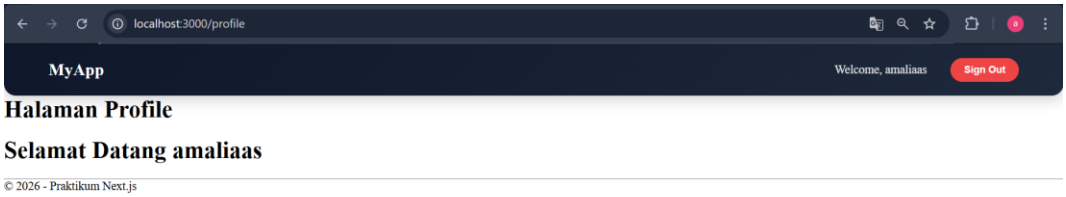
## E. Tambahkan Button Login Google

1. Modifikasi file index.tsx pada folder views/auth/login

```
    {/* Button login */}
    <button
      type="submit"
      className={style.login__form__item__button}
      disabled={isLoading}
    >
      {isLoading ? "Loading..." : "login"}
    </button>

  <br /> <br />
  <button
  onClick={() => signIn("google", {callbackUrl, redirect: false })}
  className={style.login__form__item__button}
    disabled={isLoading}
    >
      {isLoading ? "Loading..." : "sign in with google"}
  </button>
  </form>

  {/* Halaman login */}
  <br />
  <p className={style.login__form__item__text}>
    Tidak punya akun? <Link href="/auth/register">Ke Halaman Register </Link>
  </p>
</div>
```

2. Jalankan browser localhost:3000/auth/login masuk melalui sign in with google.Jika berhasil maka akan terhubung dengan akun google.






Note: data akun google tidak tersimpan dalam database

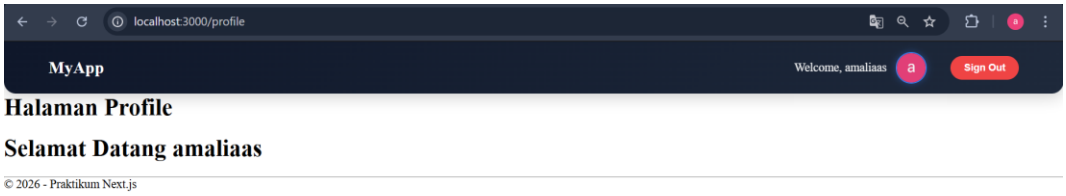
3. Menampilkan image dari google
   - Buka file index.tsx dan tambahkan code berikut

```
import styles from './navbar.module.css';
import { signIn, signOut, useSession } from 'next-auth/react';

const Navbar = () => {
  const {data}: any = useSession()
  //const { data: session } = useSession()
  // console.log("session", session)
  return (
    <div className={styles.navbar}>
      <div className={styles.navbar__brand}>
        MyApp
      </div>

      <div className={styles.navbar__right}>
        {data ? (
          <>
            <div className={styles.navbar__user}>
              Welcome, {data.user?.fullname} {/* show full name  */}
              {data.user.image && (
              <img
                src={data.user.image}
                alt={data.user.fullname}
                className={styles.navbar__user_image}
              />
            )}
            </div>
            <button
              className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`}
              onClick={() => signOut({ callbackUrl: "/" })}
            >
              Sign Out
            </button>
          </>
        ) : (
          <button
            className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`}
            onClick={() => signIn(undefined, { callbackUrl: "/profile" })}
          >
            Sign In
          </button>
        )}
      </div>
    </div>
  );
};

export default Navbar;
```

- Buka file navbar.module.css dan tambahkan code berikut

```css
/* Welcome text */
.navbar__user {
  display: flex;
  align-items: center;
  gap: 12px;
  font-size: 16px;
  font-weight: 500;
  opacity: 0.95;
}

/* Avatar image */
.navbar__user__image {
  width: 42px;
  height: 42px;
  border-radius: 50%;
  object-fit: cover;
  border: 2px solid #3b82f6;
  box-shadow: 0 0 10px rgba(59, 130, 246, 0.4);
  transition: all 0.3s ease;
}

.navbar__user__image:hover {
  transform: scale(1.1);
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.8);
}
```

- Jika berhasil maka tampillannya akan seperti berikut



## G. Simpan Data Google ke Database

- Buka file servicefirebase.ts pada folder src/utils/db/ dan tambahkan beberapa kode berikut

```
updateDoc, // menyimpan perubahan data goole ke database
```

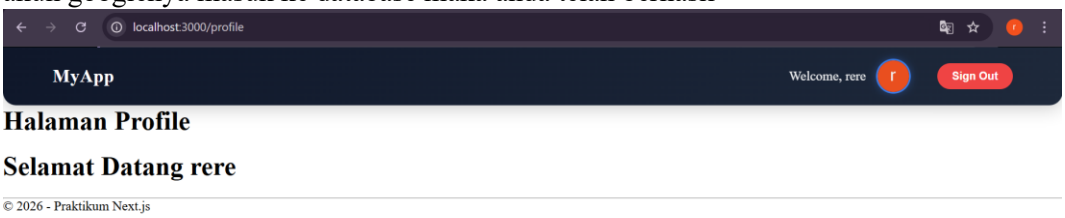
- Tambahkan juga code berikut

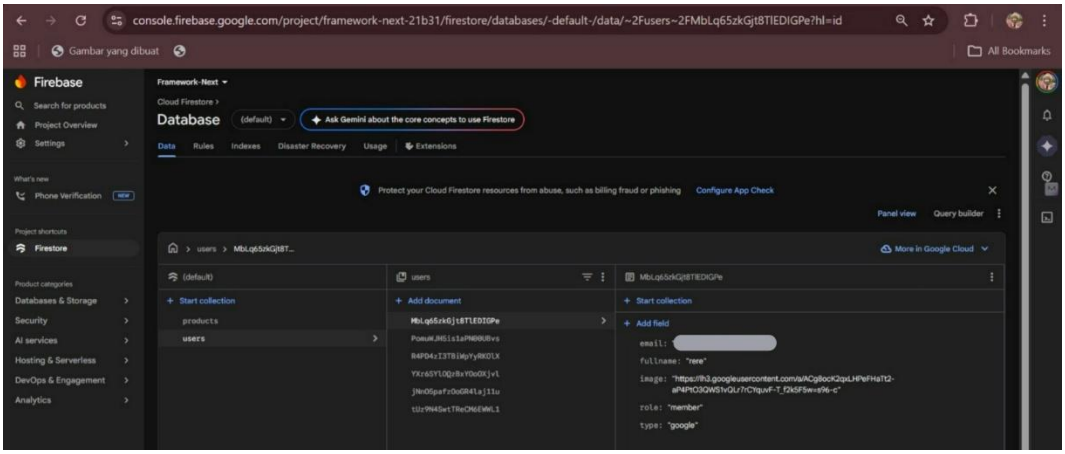
```
export async function signInWithGoogle(userData: any, callback: any) {
  try {
    // cek apakah email sudah ada di database/blm
    const q = query(
      collection(db, "users"),
      where("email", "==", userData.email),
    );

    const querySnapshot = await getDocs(q);
    const data: any = querySnapshot.docs.map((doc) => ({
      id: doc.id,
      ...doc.data(),
    }));

    if (data.length > 0) {
      // user sudah ada → ambil role lama lalu update data
      userData.role = data[0].role;
      await updateDoc(doc(db, "users", data[0].id), userData);
      callback({
        status: true,
        message: "User registered and logged in with Google",
        data: userData,
      });
    } else {
      // user baru → set role member lalu simpan ke database
      userData.role = "member";
      await addDoc(collection(db, "users"), userData);
      callback({
        status: true,
        message: "User registered and logged in with Google",
        data: userData,
      });
    }
  } catch (error: any) {
    callback({
      status: false,
      message: "Failed to register user with Google",
    });
  }
}
```

- Panggil Service di JWT Callback buka file [...nextAuth].ts

```
my-app-17 > src > pages > api > auth > [...nextauth].ts > authOptions > callbacks
7   export const authOptions: NextAuthOptions = {
50    callbacks: {
53      //   if (url.startsWith("/")) return `${baseUrl}${url}`;
54      //   return `${baseUrl}/profile`;
55      // },
56
        Windsurf: Refactor | Explain | Generate JSDoc | X
57      async jwt({ token, account, profile, user }: any) {
58        if (account?.provider === "credentials" && user) {
59          token.email = user.email;
60          token.fullname = user.fullname;
61          token.role = user.role;
62        }
63
64        //jika login dengan google, tambah informasi yg diperlukan ke token
65        if (account?.provider === "google") {
66          const data = {
67            fullname: user.name,
68            email: user.email,
69            image: user.image,
70            type: account.provider,
71          };
72
73          await signInWithGoogle(data, (result: any) => {
74            // pastikan cek result.status sesuai dengan object yang dikirim
75            if (result.status) {
76            token.fullname = result.data.fullname;
77            token.email = result.data.email;
78            token.image = result.data.image;
79            token.type = result.data.type;
80            }
81          });
82        }
83        return token;
84      },
85
```

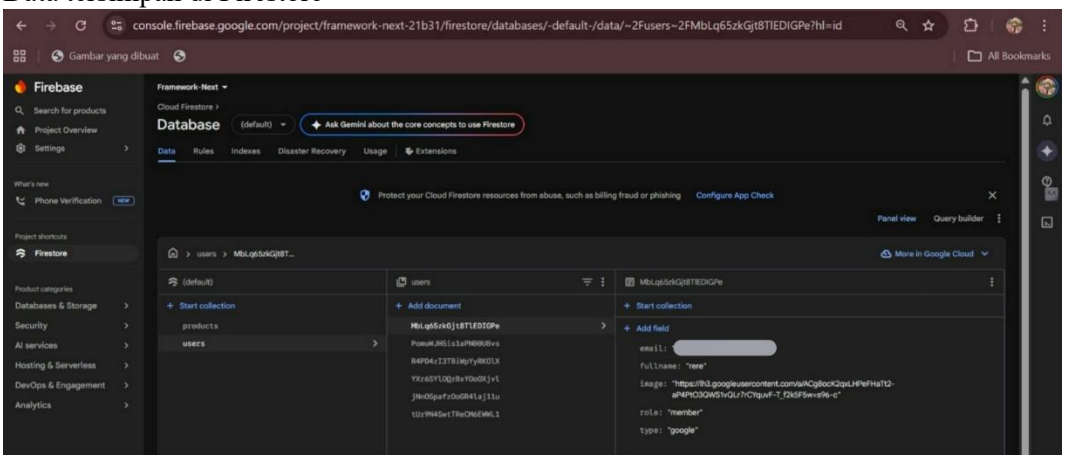
- Jalankan browser dan login menggunakan akun google setelah cek di firebase, jika data akun googlenya masuk ke database maka anda telah berhasil



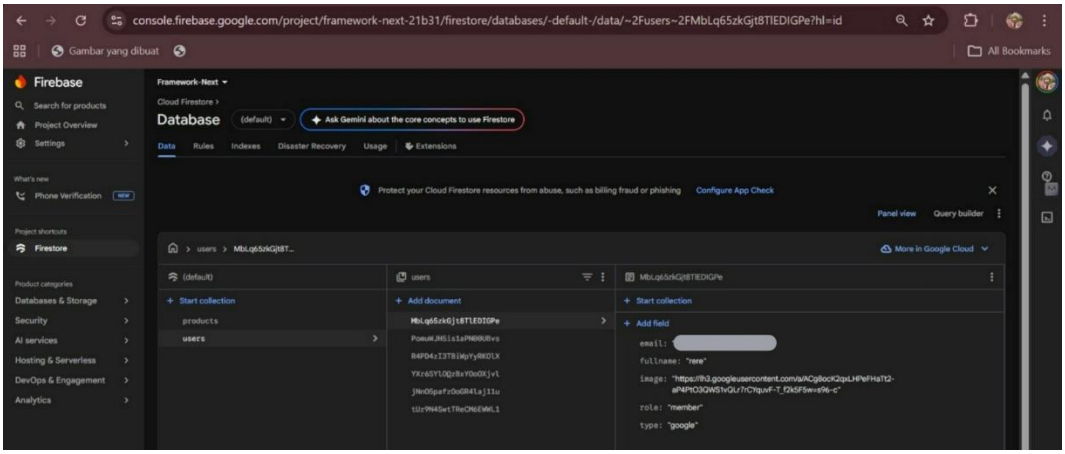



## h. Pengujian

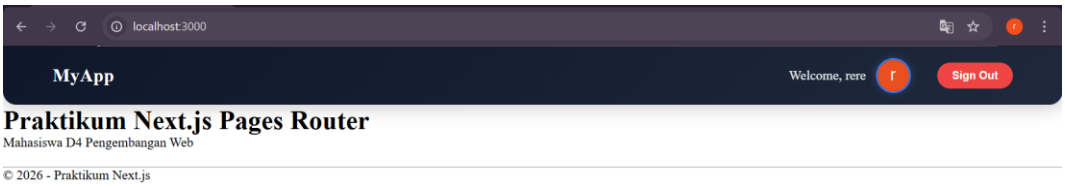
- Skenario
  Login Google pertama kali
- Hasil yang Diharapkan
  Data tersimpan di Firestore



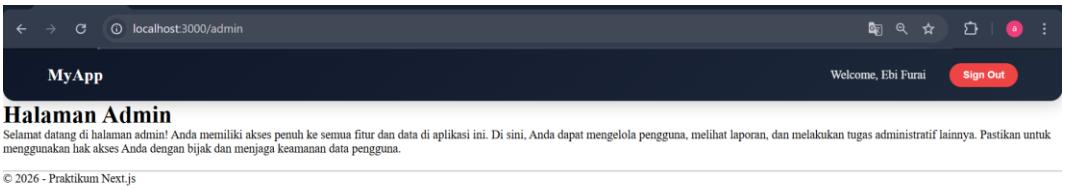

- Skenario
  Login Google kedua kali
- Hasil yang Diharapkan
  Data diupdate




- Skenario
  User role member akses /admin
- Hasil yang Diharapkan
  Redirect




- Skenario
  User role admin akses /admin
- Hasil yang Diharapkan
  Bisa masuk




- Skenario
  Avatar tampil
- Hasil yang Diharapkan
  Ya

# Analisis & Diskusi

1. Apa perbedaan login credential dan login Google?
   - Login credential menggunakan email dan password yang dibuat oleh user dan disimpan di database aplikasi. Sedangkan login Google menggunakan akun Google sebagai pihak ketiga (OAuth), sehingga proses autentikasi dilakukan oleh Google tanpa perlu menyimpan password di aplikasi.

2. Mengapa data Google tetap perlu disimpan ke database?
   - Meskipun login menggunakan Google, data user tetap perlu disimpan di database agar aplikasi dapat mengenali user tersebut, menyimpan informasi tambahan seperti role, serta mengelola data lain yang berkaitan dengan user di dalam sistem.

3. Apa fungsi JWT callback?
   - JWT callback berfungsi untuk menambahkan atau memodifikasi data yang akan disimpan di dalam token. Data seperti email, nama, dan role dapat dimasukkan ke dalam token agar bisa digunakan di berbagai bagian aplikasi tanpa perlu mengambil ulang dari database.

4. Mengapa perlu multi-role?
   - Multi-role diperlukan untuk membedakan hak akses setiap user dalam sistem. Misalnya, admin memiliki akses lebih luas dibandingkan user biasa. Dengan adanya role, sistem dapat mengatur fitur atau halaman yang boleh diakses oleh masing-masing user.

5. Apa risiko jika tidak menyimpan user ke database?
   - Jika data user tidak disimpan, aplikasi tidak dapat mengenali user secara konsisten, tidak bisa menyimpan informasi tambahan seperti role, serta sulit mengelola data yang berkaitan dengan user. Hal ini dapat menyebabkan keterbatasan fungsi dan masalah dalam pengelolaan akses.

# Tugas Mandiri

1. Tambahkan role editor
2. Buat halaman khusus editor

   Jawaban nomor 1 & 2
   Modifikasi src/Middleware/withAuth.ts
   Tambahkan pada line 5, dan line 27-29

```typescript
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

const hanyaAdmin = ["/admin"];
const hanyaEditor = ["/editor"];
export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });
      if (!token) {
        const Url = new URL("/auth/login", req.url);
        Url.searchParams.set("callbackUrl", encodeURI(req.url));
        return NextResponse.redirect(Url);
        // perlindungan untuk halaman yang membutuhkan autentikasi, jika tidak ada token maka akan diarahkan ke halaman login menggunakan middleware
      }
      if (token.role !=="admin" && hanyaAdmin.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
    }
     if (token.role !== "editor" && token.role !== "admin" && hanyaEditor.includes(pathname)) {
        return NextResponse.redirect(new URL("/", req.url));
      }
  }
    return middleware(req, next);
  };
}

```

src/middleware.ts

```typescript
import { NextMiddleware, NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import withAuth from "./Middleware/withAuth";

function mainMiddleware(request: NextRequest) {
  return NextResponse.next();
}

export default withAuth(mainMiddleware as NextMiddleware, ["/produk", "/about", "/profile", "/admin", "/editor"]);

export const config = {
  matcher: ["/produk", "/about", "/profile", "/admin", "/editor"],
};
```

src/pages/editor/index.tsx

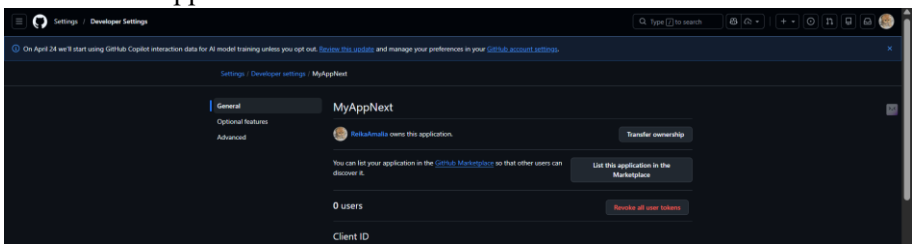
```tsx
const HalamanEditor = () => {
  return (
    <div>
      <div class="editor">
        <h1>Halaman Editor</h1>
        <p>
          Selamat datang di halaman editor! Anda memiliki akses untuk mengelola
          dan mengedit konten aplikasi. Pastikan untuk menggunakan hak akses
          Anda dengan bijak.
        </p>
      </div>
    </div>
  );
};

export default HalamanEditor;
```

3. Tambahkan provider GitHub

Pertama install provider:

```
PS D:\Pemograman_Framework\praktikum_17\my-app-17> npm install next-auth
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: 'eslint-visitor-keys@5.0.1',
npm WARN EBADENGINE   required: { node: '^20.19.0 || ^22.13.0 || >=24' },
npm WARN EBADENGINE   current: { node: 'v20.11.1', npm: '10.2.4' }
npm WARN EBADENGINE }

up to date, audited 450 packages in 12s

147 packages are looking for funding
  run `npm fund` for details

1 moderate severity vulnerability

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

Buat OAuth App baru




Masukkan ke .env.local

```
my-app-17 > .env.local
FIREBASE_API_KEY=[redacted]
FIREBASE_AUTH_DOMAIN=[redacted]
FIREBASE_PROJECT_ID=[redacted]
FIREBASE_STORAGE_BUCKET=[redacted]
FIREBASE_MESSAGING_SENDER_ID=[redacted]
FIREBASE_APP_ID=1[redacted]

REVALIDATE_TOKEN=12345678


# gunakan website https://generate-secret.vercel.app/ untuk membuat sec
# lalu masukkan ke dalam variabel NEXTAUTH_SECRET
NEXTAUTH_SECRET=[redacted]

GOOGLE_CLIENT_ID=[redacted]
GOOGLE_CLIENT_SECRET=[redacted]

GITHUB_CLIENT_ID=[redacted]
GITHUB_CLIENT_SECRET=[redacted]
```

Modifikasi src/pages/api/auth/[...nextauth].ts

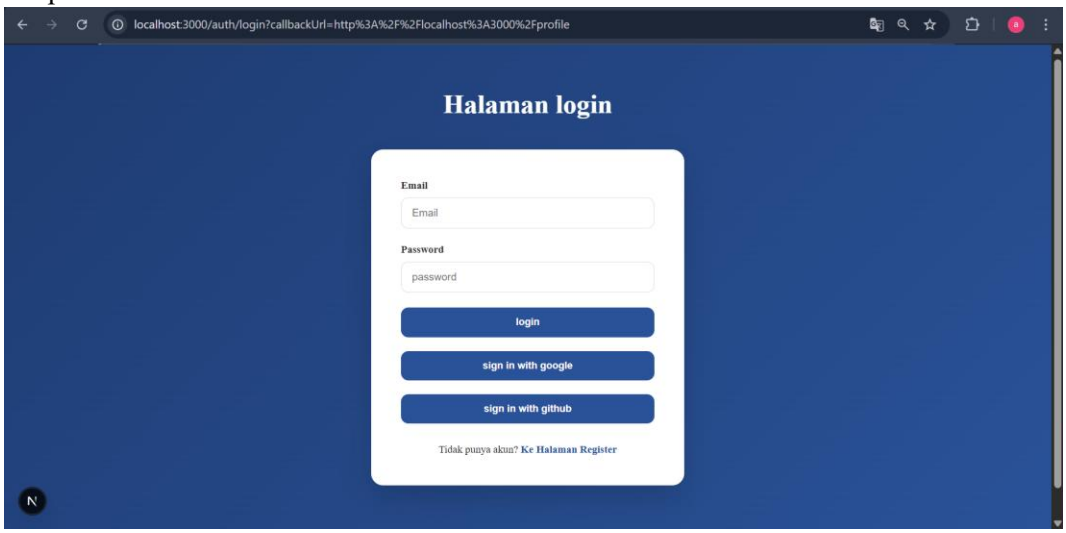
```python
import { signIn, signInWithGoogle } from "@/utils/db/servicefirebase";
import NextAuth, { NextAuthOptions } from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";
import bcrypt from "bcrypt";
import GoogleProvider from "next-auth/providers/google";
import GithubProvider from "next-auth/providers/github"; // import GitHub provider

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,

  providers: [
    CredentialsProvider({
      name: "credentials",
      credentials: {
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        if (!credentials?.email || !credentials.password) return null;
        const user: any = await signIn(credentials.email);
        if (user) {
          const isPasswordValid = await bcrypt.compare(
            credentials.password,
            user.password,
          );
          if (isPasswordValid) {
            return {
              id: user.id,
              email: user.email,
              fullname: user.fullname,
              role: user.role,
            };
          }
        }
        return null;
      },
    }),
    GoogleProvider({
      clientId: process.env.GOOGLE_CLIENT_ID || "",
      clientSecret: process.env.GOOGLE_CLIENT_SECRET || "",
    }),
    // GitHub provider
    GithubProvider({
      clientId: process.env.GITHUB_CLIENT_ID || "",
      clientSecret: process.env.GITHUB_CLIENT_SECRET || "",
    }),
  ],

  callbacks: {

    async jwt({ token, account, profile, user }: any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
        token.role = user.role;
      }

      if (account?.provider === "google") {
        const data = {
          fullname: user.name,
          email: user.email,
          image: user.image,
          type: account.provider,
        };
        await signInWithGoogle(data, (result: any) => {
          if (result.status) {
            token.fullname = result.data.fullname;
            token.email = result.data.email;
            token.image = result.data.image;
            token.type = result.data.type;
            token.role = result.data.role; // tambahkan role
          }
        });
      }

      // handler untuk GitHub
      if (account?.provider === "github") {
        const data = {
          fullname: user.name,
          email: user.email,
          image: user.image,
          type: account.provider,
        };
        await signInWithGoogle(data, (result: any) => {
          // reuse signInWithGoogle
          if (result.status) {
            token.fullname = result.data.fullname;
            token.email = result.data.email;
            token.image = result.data.image;
            token.type = result.data.type;
            token.role = result.data.role;
          }
        });
      }

      return token;
    },

    async session({ session, token }: any) {
      if (token.email) session.user.email = token.email;
      if (token.fullname) session.user.fullname = token.fullname;
      if (token.image) session.user.image = token.image;
      if (token.role) session.user.role = token.role;
      if (token.type) session.user.type = token.type;
      return session;
    },
  },


  pages: {
    signIn: "/auth/login",
  },
};

export default NextAuth(authOptions);
```

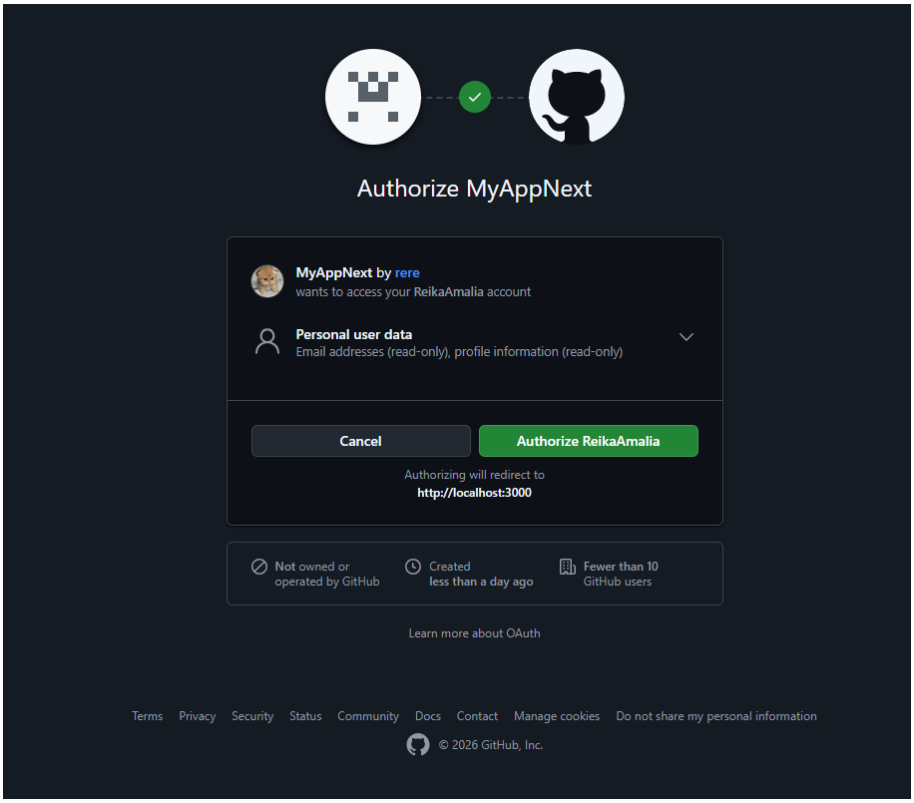
Tambahkan button GitHub di src/views/auth/login/index.tsx

my-app-17 > src > views > auth > login > index.tsx > TampilanLogin

```
7   TampilanLogin = () => {
109     {/* Button login */}
110     <button
111       type="submit"
112       className={style.login__form__item__button}
113       disabled={isLoading}
114     >
115       {isLoading ? "Loading..." : "login"}
116     </button>
117
118   <br /> <br />
119   <button
120   onClick={() => signIn("google", {callbackUrl, redirect: false })}
121   className={style.login__form__item__button}
122     disabled={isLoading}
123     >
124       {isLoading ? "Loading..." : "sign in with google"}
125   </button>
126
127    <br /> <br />
128     {/* button GitHub */}
129     <button
130       onClick={() => signIn("github", { callbackUrl, redirect: false })}
131       className={style.login__form__item__button}
132       disabled={isLoading}
133     >
134       {isLoading ? "Loading..." : "sign in with github"}
135     </button>
136   </form>
137
138   {/* Halaman login */}
```

Output:






4. Refactor service agar reusable
   Tambahkan fungsi reusable di src/utils/db/servicefirebase.ts

```
import {
  getFirestore,
  collection,
  getDocs,
  getDoc,
  doc,
  query,
  addDoc,
  where,
  updateDoc,
} from "firebase/firestore";
import app from "./firebase";
import bcrypt from "bcrypt";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
  const snapshot = await getDocs(collection(db, collectionName));
  const data = snapshot.docs.map((doc) => ({
    id: doc.id,
    ...doc.data(),
  }));
  return data;
}

export async function retrieveDataByID(collectionName: string, id: string) {
  const snapshot = await getDoc(doc(db, collectionName, id));
  const data = snapshot.data();
  return data;
}

// fungsi reusable untuk cek user by email
// dipakai oleh signIn, signUp, dan signInWithGoogle agar tidak perlu tulis query berulang
export async function getUserByEmail(email: string) {
  const q = query(collection(db, "users"), where("email", "==", email));
  const querySnapshot = await getDocs(q);
  const data = querySnapshot.docs.map((doc) => ({
    id: doc.id,
    ...doc.data(),
  }));
  return data;
}

// fungsi reusable untuk simpan user baru ke database
export async function saveUser(userData: any) {
  return await addDoc(collection(db, "users"), userData);
}

// fungsi reusable untuk update data user di database
export async function updateUser(id: string, userData: any) {
  return await updateDoc(doc(db, "users", id), userData);
}

export async function signIn(email: string) {
  // refactor: pakai getUserByEmail agar tidak duplikasi query
  const data = await getUserByEmail(email);
  if (data) {
    return data[0];
  } else {
    return null;
  }
}

export async function signUp(
  userData: {
    email: string;
    fullname: string;
    password: string;
    role?: string;
  },
  callback: Function,
) {
  // refactor: pakai getUserByEmail agar tidak duplikasi query
  const data = await getUserByEmail(userData.email);

  if (data.length > 0) {
    callback({
      status: "error",
      message: "Email already exists",
    });
  } else {
    userData.password = await bcrypt.hash(userData.password, 10);
    userData.role = "user";
    // refactor: pakai saveUser agar tidak duplikasi addDoc
    await saveUser(userData)
      .then(() => {
        callback({
          status: "success",
          message: "User registered successfully",
        });
      })
      .catch((error) => {
        callback({
          status: "error",
          message: error.message,
        });
      });
  }
}

// fungsi untuk simpan/update data user Google ke Firestore
export async function signInWithGoogle(userData: any, callback: any) {
  try {
    // refactor: pakai getUserByEmail agar tidak duplikasi query
    const data: any = await getUserByEmail(userData.email);

    if (data.length > 0) {
      // user sudah ada → ambil role lama lalu update data
      userData.role = data[0].role;
      // refactor: pakai updateUser agar tidak duplikasi updateDoc
      await updateUser(data[0].id, userData);
      callback({
        status: true,
        message: "User registered and logged in with Google",
        data: userData,
      });
    } else {
      // user baru → set role member lalu simpan ke database
      userData.role = "member";
      // refactor: pakai saveUser agar tidak duplikasi addDoc
      await saveUser(userData);
      callback({
        status: true,
        message: "User registered and logged in with Google",
        data: userData,
      });
    }
  } catch (error: any) {
    callback({
      status: false,
      message: "Failed to register user with Google",
    });
  }
}
```

5. Gunakan next/image untuk optimasi avatar
   Tambahkan domain di next.config.js

```javascript
/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    domains: [
      "lh3.googleusercontent.com", // Google
      "avatars.githubusercontent.com", // GitHub
    ],
  },
};

module.exports = nextConfig;
```

   Update src/components/navbar/index.tsx

```
1  import styles from './navbar.module.css';
2  import { signIn, signOut, useSession } from 'next-auth/react';
3  import Image from 'next/image';
4
5  const Navbar = () => {
6    const {data}: any = useSession()
7    //const { data: session } = useSession()
8    // console.log("session", session)
9    return (
10     <div className={styles.navbar}>
11       <div className={styles.navbar__brand}>
12         MyApp
13       </div>
14
15       <div className={styles.navbar__right}>
16         {data ? (
17           <>
18             <div className={styles.navbar__user}>
19               Welcome, {data.user?.fullname || data.user?.name} {/* show full name  */}
20               {data.user.image && (
21               <Image
22                 src={data.user.image}
23                 alt="avatar"
24                 width={32}
25                 height={32}
26                 referrerPolicy="no-referrer"
27                 className={styles.navbar__user__image}
28               />
29             )}
30             </div>
31             <button
32               className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`}
33               onClick={() => signOut({ callbackUrl: "/" })}
34             >
35               Sign Out
36             </button>
37           </>
38         ) : (
39           <button
40             className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`}
41             onClick={() => signIn(undefined, { callbackUrl: "/profile" })}
42           >
43             Sign In
44           </button>
45         )}
46       </div>
47     </div>
48   );
49 };
50
51 export default Navbar;
```